

15 Backup & Recovery

No operational concern in PostgreSQL carries higher stakes than backup and recovery. Everything else — replication, query tuning, connection pooling — is about performance and availability. Backups are about whether your data exists at all after a catastrophic failure. This folder covers the full spectrum: logical dumps with `pg_dump`, physical base backups with `pg_basebackup`, continuous WAL archiving for point-in-time recovery, enterprise backup management with `pgBackRest` and WAL-G, and the monitoring practices required to know that backups will actually work when you need them.

Contents

Filename	What It Covers	Key Tools
01_backup_fundamentals.md	RPO/RTO concepts, backup types, strategy selection	<code>pg_dump</code> , <code>pg_restore</code>
02_logical_backups.md	<code>pg_dump</code> flags, custom vs directory format, parallel dump/restore	<code>pg_dump</code> , <code>pg_restore</code> , <code>pg_dumpall</code>
03_physical_basebackup.md	<code>pg_basebackup</code> mechanics, streaming vs tar format, replication slots	<code>pg_basebackup</code>
04_wal_archiving.md	<code>archive_command</code> setup, WAL segment lifecycle, <code>archive_cleanup_command</code>	PostgreSQL <code>archive_command</code>
05_pitr_recovery.md	Point-in-time recovery to timestamp/LSN/XID, <code>recovery.signal</code> , recovery targets	<code>postgresql.conf</code> recovery settings
06_pgbackrest.md	<code>pgBackRest</code> configuration, full/diff/incr backups, parallel restore, retention	<code>pgBackRest</code>
07_walg.md	WAL-G setup for S3/GCS/Azure, compression, delta backups	WAL-G
08_backup_monitoring.md	<code>pg_verifybackup</code> , <code>pg_stat_archiver</code> , restore drills, age alerting	<code>pg_verifybackup</code> , <code>pgBackRest</code> info

RPO/RTO Quick Reference

Backup Type	Typical RPO	Typical RTO	Complexity
<code>pg_dump</code> (daily)	24 hours	1–4 hours (depends on DB size)	Low
<code>pg_dump</code> (hourly)	1 hour	1–4 hours	Low–Medium
<code>pg_basebackup</code> + WAL archive	Near-zero (last WAL segment ~16 MB)	30 min – 4 hours	Medium
<code>pgBackRest</code> full + incremental + WAL	Near-zero	15 min – 2 hours	Medium–High

WAL-G + S3 + continuous archiving	Near-zero (seconds with streaming)	20 min – 2 hours	Medium–High
Streaming replication (hot standby)	Near-zero (seconds of lag)	Minutes (failover, no restore needed)	High

RPO = Recovery Point Objective (maximum data loss tolerance). RTO = Recovery Time Objective (maximum acceptable time to restore service). Values assume typical hardware; actual times depend on database size, storage speed, and network bandwidth.

Learning Path

- 1. Start with fundamentals (01)** — Understand RPO, RTO, and how to choose between logical and physical backups before touching any tools. Choosing the wrong backup strategy is more expensive than not backing up at all, because it creates false confidence.
- 2. Learn logical backups (02)** — `pg_dump` is the tool everyone uses first and it is more nuanced than it appears. Learn custom format, parallel restore (`-j`), and the limitations (no point-in-time recovery, no transaction consistency across databases in `pg_dumpall`).
- 3. Learn physical backups (03–04)** — `pg_basebackup` + WAL archiving is the foundation for PITR. Master these before moving to `pgBackRest`, since `pgBackRest` automates what these tools do manually.
- 4. Practice PITR (05)** — Perform a full PITR drill manually before using `pgBackRest`'s restore. Understanding the recovery signal file, `recovery_target_time`, and timeline branching will save hours when debugging a `pgBackRest` restore gone wrong.
- 5. Learn `pgBackRest` or WAL-G (06–07)** — Choose the tool that fits your environment. `pgBackRest` is more feature-complete and integrates tightly with PostgreSQL. WAL-G is lighter and excels in cloud-native (S3/GCS) environments. Most new setups choose one of these rather than bare `archive_command`.
- 6. Implement monitoring last but do not skip it (08)** — After setting up backups, monitoring is what makes them real. A backup without monitoring is an unverified hypothesis.

Prerequisites

- **SQL and `psql` basics** — You need to run queries and connect to PostgreSQL to follow the monitoring examples
- **Linux command line** — Backup tools are shell-driven; comfort with `bash`, `cron`, and filesystem commands is required
- **PostgreSQL server configuration** — Know how to edit `postgresql.conf` and `pg_hba.conf` and reload/restart the server
- **Transactions and WAL basics** — Understanding that PostgreSQL writes ahead-of-change to WAL (Write-Ahead Log) before modifying data files is essential for understanding why WAL archiving enables point-in-time recovery

Key Tools Summary

pg_dump is PostgreSQL's built-in logical backup tool. It produces a consistent snapshot of a single database (or selected schemas/tables) in plain SQL, custom binary, directory, or tar format. Custom format (`-Fc`) is the most flexible: it supports parallel restore with `-j N` workers and selective object restoration with `pg_restore -t tablename` . Limitations: `pg_dump` cannot back up a database that is being modified during the dump in a way that exceeds transaction duration (very long-running dumps may fail on highly-active databases), and it cannot perform point-in-time recovery. Use `pg_dump` for: schema backups, migrating between PostgreSQL versions, logical replication of subsets of data, and as a complement to physical backups for quick schema recovery.

pg_basebackup is PostgreSQL's built-in physical backup tool. It connects to a running primary (or replica) via the replication protocol and streams a consistent copy of the data directory as a base backup, including the WAL needed to make the backup self-consistent. It supports plain (`-Fp`) and tar (`-Ft`) format, and streaming WAL simultaneously with the base backup (`-Xs`) to minimize the required WAL retention. The output can be used directly as a standby data directory (with `-R` to generate `standby.signal` and recovery settings) or as a starting point for PITR combined with WAL archives. `pg_basebackup` is simple but has limitations: no incremental backups, no built-in compression (without `--compress` flag), and no backup retention management.

pgBackRest is a comprehensive PostgreSQL backup and restore solution with full, differential, and incremental backup types, parallel backup and restore, built-in compression (gzip/lz4/zstd/bz2), multiple repository support (local, S3, GCS, Azure), backup encryption, backup verification, and asynchronous WAL archiving. Its `pgbackrest info` command provides detailed backup catalog and WAL archive status, and `pgbackrest check` performs end-to-end verification of the archive pipeline. `pgBackRest` is the recommended choice for production deployments that require flexible retention policies, cloud storage integration, and operational simplicity.

WAL-G is a lightweight, open-source WAL archiving and base backup tool designed for cloud-native environments. It supports S3, GCS, Azure Blob, and local filesystem as backup destinations, with built-in delta backup capability (copies only changed blocks within data files using page checksums), LZ4/LZMA/zstd/brotli compression, and parallel upload/download. WAL-G is typically configured as the `archive_command` and `restore_command` , replacing the manual `cp/aws s3 cp` approach. It excels in containerized and cloud-native PostgreSQL deployments where simplicity and cloud-storage integration matter more than the full feature set of `pgBackRest`.

Related Folders

- **13_Replication_HA** — Streaming replication and high availability. Physical backups taken from a replica (`pg_basebackup` from a standby) reduce primary load. Replication slots interact with WAL retention and can block `VACUUM` if abandoned.
- **12_Production_PostgreSQL** — Production operations and monitoring. Backup age alerts, archive health metrics, and restore drill documentation belong in the production runbook maintained alongside the material in that folder.